

ALGORITHME pipeline_nlp_malagasy(texte)

ENTREE: texte brut

SORTIE: analyse complète

resultats = dictionnaire_vide()

// 1. Prétraitement

texte_nettoye = nettoyer_texte(texte)

tokens = tokeniser(texte_nettoye)

// 2. Analyse en parallèle (approches hybrides)

EN PARALLÈLE:

// Approche symbolique

erreurs_phonotactiques = verifier_regles_linguistiques(tokens)

structure_mots = analyser_structure_mots(tokens)

// Approche statistique

suggestions_ngram = predire_avec_ngrams(tokens)

similarites = calculer_similarites_statistiques(tokens)

// Approche par dictionnaire

corrections_dict = verifier_dictionnaire(tokens)

traductions = traduire_mots_cles(tokens)

FIN PARALLÈLE

// 3. Fusion des résultats

```
corrections_finales = fusionner_corrections(  
    erreurs_phonotactiques,  
    corrections_dict,  
    similarites  
)
```

// 4. Analyse sémantique basique

```
sentiment = analyser_sentiment_basique(tokens)  
themes = detecter_themes(tokens)
```

// 5. Construction de la réponse

```
resultats['corrections'] = corrections_finales  
resultats['suggestions'] = suggestions_ngram  
resultats['traductions'] = traductions  
resultats['sentiment'] = sentiment  
resultats['structure'] = structure_mots  
resultats['themes'] = themes
```

// 6. Post-traitement

```
resultats['confiance'] = calculer_confiance(resultats)  
resultats['explications'] = generer_explications(resultats)
```

RETOURNER resultats

FIN ALGORITHME

ALGORITHME fusionner_corrections(erreurs_regles, erreurs_dict, similarites)

ENTREE: trois listes de corrections potentielles

SORTIE: liste de corrections fusionnées

```
corrections_fusionnees = liste_vide()
```

// Pour chaque mot problématique

```
mots_problemes = union(erreurs_regles.mots, erreurs_dict.mots)
```

POUR CHAQUE mot DANS mots_problemes FAIRE

```
    correction = dictionnaire_vide()
```

```
    correction['mot'] = mot
```

// Combinaison des sources

SI mot ∈ erreurs_regles ALORS

```
    correction['type_regle'] = erreurs_regles[mot].type
```

```
    correction['severite'] = 'HAUTE'
```

FIN SI

SI mot ∈ erreurs_dict ALORS

```
    correction['type_dict'] = 'HORS_DICTIONNAIRE'
```

```
    correction['suggestions_dict'] = erreurs_dict[mot].suggestions
```

FIN SI

SI mot ∈ similarites ALORS

```
    correction['alternatives'] = similarites[mot].top(3)
```

FIN SI

// Calcul du score de confiance

score = calculer_score_confiance(correction)

correction['confidence'] = score

// Seuil de filtrage

SI score > SEUIL_CONFIANCE ALORS

 ajouter(corrections_fusionnees, correction)

FIN SI

FIN POUR

RETOURNER trier_par_severite(corrections_fusionnees)

FIN ALGORITHME